

Mohamed Kane

Professor Swart

ENGMT 5400

23 March 2026

Rock-Paper-Scissors CNN on ESP32-S3

1. Design Description

This project implements a convolutional neural network (CNN) that classifies rock, paper, and scissors hand signs in real time using the ESP32-S3 microcontroller. The system uses the onboard camera to capture images, preprocesses them on-device, runs inference using a custom MicroPython CNN engine, and streams predictions back to a laptop over WiFi.

1.1 System Architecture

The end-to-end pipeline is as follows:

- The ESPcamera captures a 96x96 pixel grayscale BMP image.
- The image is resized to 32x32 pixels using the `resize_96x96_to_32x32_and_threshold()` function from `image_preprocessing.py`, with a threshold of 128 applied to create a binary black-and-white image.
- The BMP header is stripped to extract raw pixel data.
- The pixel data is passed to the CNN inference engine (`cnn_inference.py`), which runs a full forward pass using weights loaded from `weights.json`.
- The predicted class label and probabilities are sent over a TCP socket to the laptop server.
- The laptop server prints the prediction in real time.

1.2 Data Collection

Training data was collected directly from the ESP32-S3 camera using a socket streaming pipeline. A MicroPython script on the ESP captured images and streamed them over WiFi to a Python server on the laptop, which saved them into labeled folders (rock/, paper/, scissors/). Images were collected in two sessions under different lighting conditions to improve model robustness. A total of approximately 900+ images were collected across three classes.

1.3 Training

Training data was captured by using the ESP to get images streamed over WiFi and saved locally into different folders labeled rock, paper, and scissors with 500 images per group. The CNN was trained using TensorFlow with the tensorflow-macos backend. Images were loaded in grayscale, resized to 32x32, normalized to [0, 1], and split 80/20 for training and validation. The model was trained for 40

epochs using the Adam optimizer with sparse categorical cross-entropy loss, achieving 100% training accuracy and 93.89% test accuracy on the laptop.

2. CNN Architecture

The model uses a compact CNN architecture designed to balance accuracy with memory constraints on the ESP32-S3. The architecture consists of three convolutional blocks followed by two fully connected layers, with a total of 30,563 trainable parameters.

The CNN consists of 10 layers in total: three convolutional layers, three MaxPooling layers, a Flatten layer, two Dense layers, and a Dropout layer, with 30,563 trainable parameters. The three convolutional layers are the core of the architecture — each learns increasingly complex visual features from the input image. The remaining layers handle dimensionality reduction, regularization, and final classification.

1. **Input (32x32x1):** The network receives a single-channel grayscale image of 32x32 pixels, normalized to values between 0 and 1.
2. **Conv2D — 16 filters (32x32x16):** 16 learnable 3x3 filters slide across the input, each producing a feature map highlighting a specific low-level pattern such as horizontal edges, vertical edges, or brightness gradients. Same padding keeps the output at 32x32. ReLU activation sets negative values to zero.
3. **MaxPooling2D (16x16x16):** A 2x2 window retains only the maximum value in each region, halving the spatial dimensions from 32x32 to 16x16 and making the model less sensitive to exact feature position.
4. **Conv2D — 32 filters (16x16x32):** 32 filters combine information across the 16 low-level feature maps to detect mid-level patterns such as finger outlines, curved shapes, and palm boundaries.
5. **MaxPooling2D (8x8x32):** Spatial dimensions reduced from 16x16 to 8x8, compressing the representation while retaining the most prominent features.
6. **Conv2D — 32 filters (8x8x32):** Operates on the compressed 8x8 maps to learn high-level patterns characteristic of each hand sign: the closed fist of rock, the flat spread of paper, or the two-finger extension of scissors.
7. **MaxPooling2D (4x4x32):** Final pooling reduces spatial dimensions to 4x4, producing a compact feature representation encoding the most important structural information.
8. **Flatten (512):** The 4x4x32 tensor is flattened into a single vector of 512 values, converting spatial feature maps into a format suitable for the fully connected layers.
9. **Dense — 32 units, ReLU (32):** Each of the 32 neurons receives input from all 512 features and learns which combinations are most predictive of each class.
10. **Dropout (0.3):** During training, 30% of neurons are randomly disabled on each forward pass, preventing over-reliance on any single neuron and reducing overfitting.

11. Dense — 3 units, Softmax (3): The output layer produces three probability values — one per class. The class with the highest probability is taken as the final prediction: rock, paper, or scissors.

3. Potential Improvements

Several improvements could increase classification accuracy and system performance:

3.1 Real-Time Video Classification

The most impactful improvement would be enabling true real-time video classification — where the camera streams continuous frames and predictions update live on screen. Currently, inference takes several seconds per frame due to the pure MicroPython convolution engine. To achieve video-rate classification, inference speed would need to drop to under 100ms per frame.

3.2 Model and Data

- Collect more diverse training data across different lighting conditions, backgrounds, and hand sizes to improve generalization. Currently, all data was collected at the same time and place, only using my hand.
- Apply data augmentation (rotations, flips, brightness shifts) to increase dataset variety without additional collection.

3.3 Runtime

The current inference engine processes frames in python loops performing convolution under 3 layers. This takes a long time to achieve the predictions. This process could be improved by getting a faster runtime by possibly reducing model/layer complexity or changing various modules within the model to optimize speed.

4. Code Attribution

The following code was written with assistance from Claude (Anthropic, claude-sonnet-4-6):

- `cnn_inference.py` — custom MicroPython CNN forward-pass inference engine
- `realtime_classify.py` — ESP32 real-time classification and streaming script
- `realtime_server.py` — laptop server for receiving and displaying predictions
- `stream_client.py` — ESP32 image streaming client
- `stream_server.py` — laptop image collection server
- `train_cnn.py` / `retrain_and_convert.py` — model training scripts
- `export_weights.py` / `convert_tflite.py` — model export and conversion scripts

The following code was provided by the course instructor via canvas:

- `image_preprocessing.py` — BMP resizing, thresholding, and header stripping functions
- `camera_test.py` — camera initialization example

- `Wifi.py` — WiFi connection helper
- `firmware.bin` — custom MicroPython firmware with camera support for XIAO ESP32-S3